

Branch and Bound

Introduction

- In backtracking, we used depth-first search with pruning to traverse the (virtual) state space. We can achieve better performance for many problems using a breadth-first search with pruning. This approach is known as branch-and-bound
- Branch and Bound is a general search method.
- Starting by considering the root problem (the original problem with the complete feasible region), the lower bounding and upper-bounding procedures are applied to the root problem.
- If the bounds match, then an optimal solution has been found and the procedure terminates.
- Otherwise, the feasible region is divided into two or more regions, these subproblems partition the feasible region.

Introduction

- The algorithm is applied recursively to the subproblems. If an optimal solution is found to a sub problem, it is a feasible solution to the full problem, but not necessarily globally optimal.
- If the lower bound for a node exceeds the best known feasible solution, no globally optimal solution can exist in the subspace of the feasible region represented by the node. Therefore, the node can be removed from consideration.
- The search proceeds until all nodes have been solved or pruned, or until some specified threshold is met between the best solution found and the lower bounds on all unsolved subproblems.

State Space Pruning

A search along a path is terminated if

- The value of the node's bound is not better than the value of the best solution seen so far
- Constraints of solution already violated, hence node represents no feasible solution
- The subset of feasible solutions represented by the node consists of a single point (and hence no further choices can be made)

Branch and Bound - Examples

- 0-1 Knapsack problem
 - Can be solved by other approaches but Branch and Bound solves with a better complexity equivalent to dynamic programming
- Travelling Salesman's problem
- N-Queens problem
 - Backtracking approach solves better than Branch and Bound
- Branch and Bound best suits the combinatorial optimization problems (i.e.) optimization problems for which order of items picked doesn't matter...

0-1 Knapsack Problem

- Given two integer arrays **val[0..n-1]** and **wt[0..n-1]** that represent values and weights associated with n items respectively. Find out the maximum value subset of val[] such that sum of the weights of this subset is smaller than or equal to Knapsack capacity W.
 - A **Greedy** approach is to pick the items in decreasing order of value per unit weight. The Greedy approach works only for fractional knapsack problem and may not produce correct result for 0/1 knapsack.
 - We can use **Dynamic Programming (DP)** for 0/1 Knapsack problem. In DP, we use a 2D table of size $n \times W$. The **DP Solution doesn't work if item weights are not integers**.
 - Since DP solution doesn't always work, a solution is to use **Brute Force**. With n items, there are 2^n solutions to be generated, check each to see if they satisfy the constraint, save maximum solution that satisfies constraint.
 - We can use **Backtracking** to optimize the Brute Force solution. We can do better (than backtracking) if we know a bound on best possible solution subtree rooted with every node. So we compute bound (best solution) for every node and compare the bound with current best solution before exploring the node.

0-1 Knapsack – Branch and Bound

At each node, record the following

- total weight w of the selection
- the total value v of this selection
- Upper bound b
 - $b = v + (W - w) (v_{i+1} / w_{i+1})$
 - v is the total value of items already selected
 - $W - w$ is the remaining capacity of knapsack
 - v_{i+1} / w_{i+1} is the best per unit payoff among the remaining items

Example 1

- Solve the given 0-1 knapsack problem using branch and bound method. Capacity of the knapsack is 10.

Item	Weight	Value
1	4	40
2	7	42
3	5	25
4	3	12

Solution:

Step 1: Calculate the upper and the lower bound

Step 2: Calculate the value index and sort the itemset in descending order based on value index

Step 3: Construct the state space tree using the approach

Example 1

- Calculate the value index and sort in descending order

already
sorted

Item	Weight	Value
1	4	40
2	7	42
3	5	25
4	3	12



Item	Weight	Value	Value Index
1	4	40	10
2	7	42	6
3	5	25	5
4	3	12	4

- Upper bound:

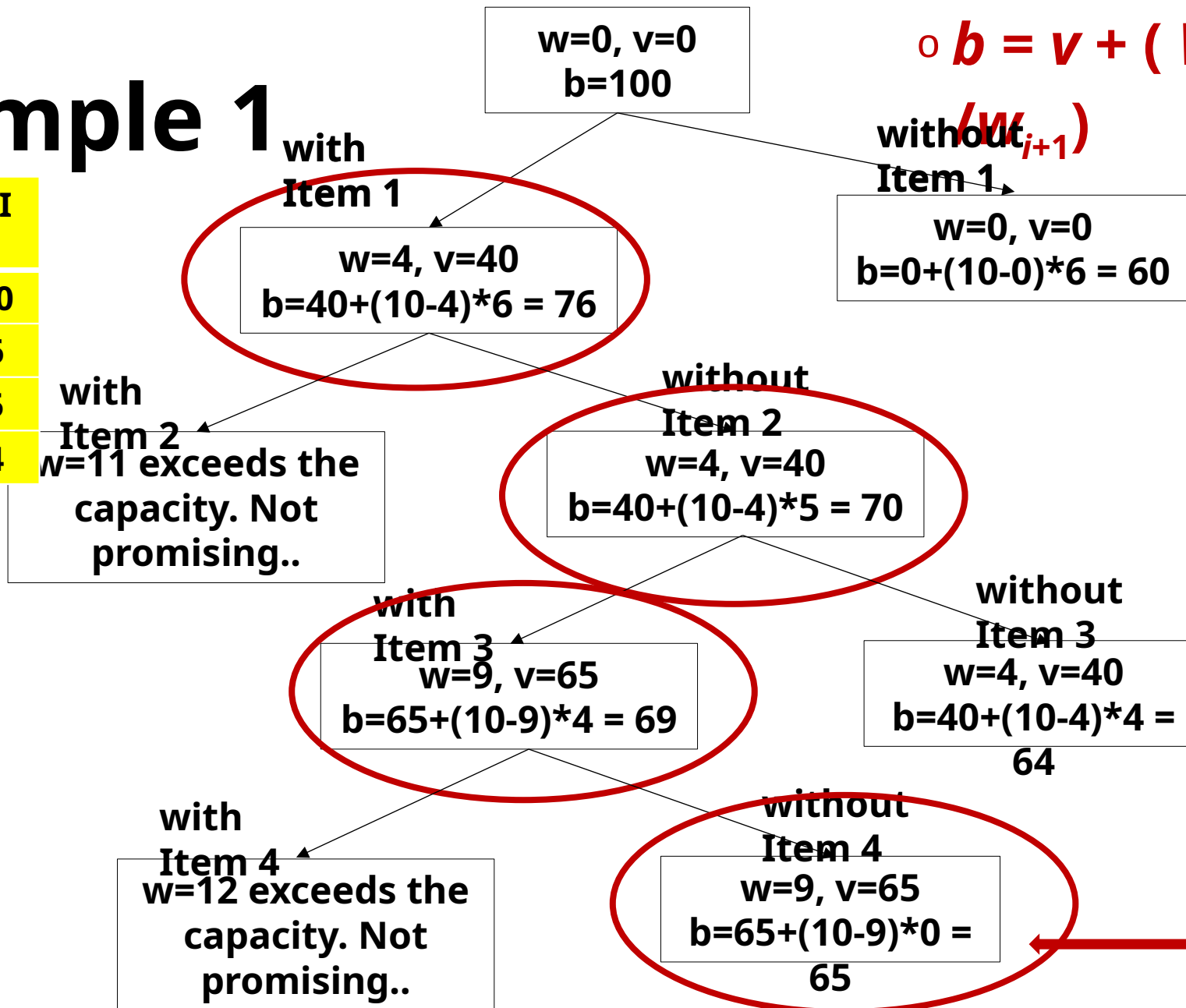
- $v=0$, $W=10$, $w=0$
- $b=0+(10-0)(10)=100$

Upper bound b

- $b = v + (W - w)(v_{i+1}/w_{i+1})$
- v is the total value of items already selected
- $W-w$ is the remaining capacity of knapsack
- v_{i+1}/w_{i+1} is the best per unit payoff among the remaining items

Example 1

Item	w	v	VI
1	4	40	10
2	7	42	6
3	5	25	5
4	3	12	4



$$b = v + (W - w) (v_{i+1} / w_{i+1})$$

Final Solution (as item list is empty):
Profit= 65
Items picked =

Example 2

- A thief enters a house for robbing it. He can carry a maximal weight of 25 kg into his bag. There are 4 items in the house with the following weights and values. What items should thief take if he either takes the item completely or leaves it completely?

Item	Weight (KG)	Value (\$)
1	24	24
2	10	18
3	10	18
4	7	10



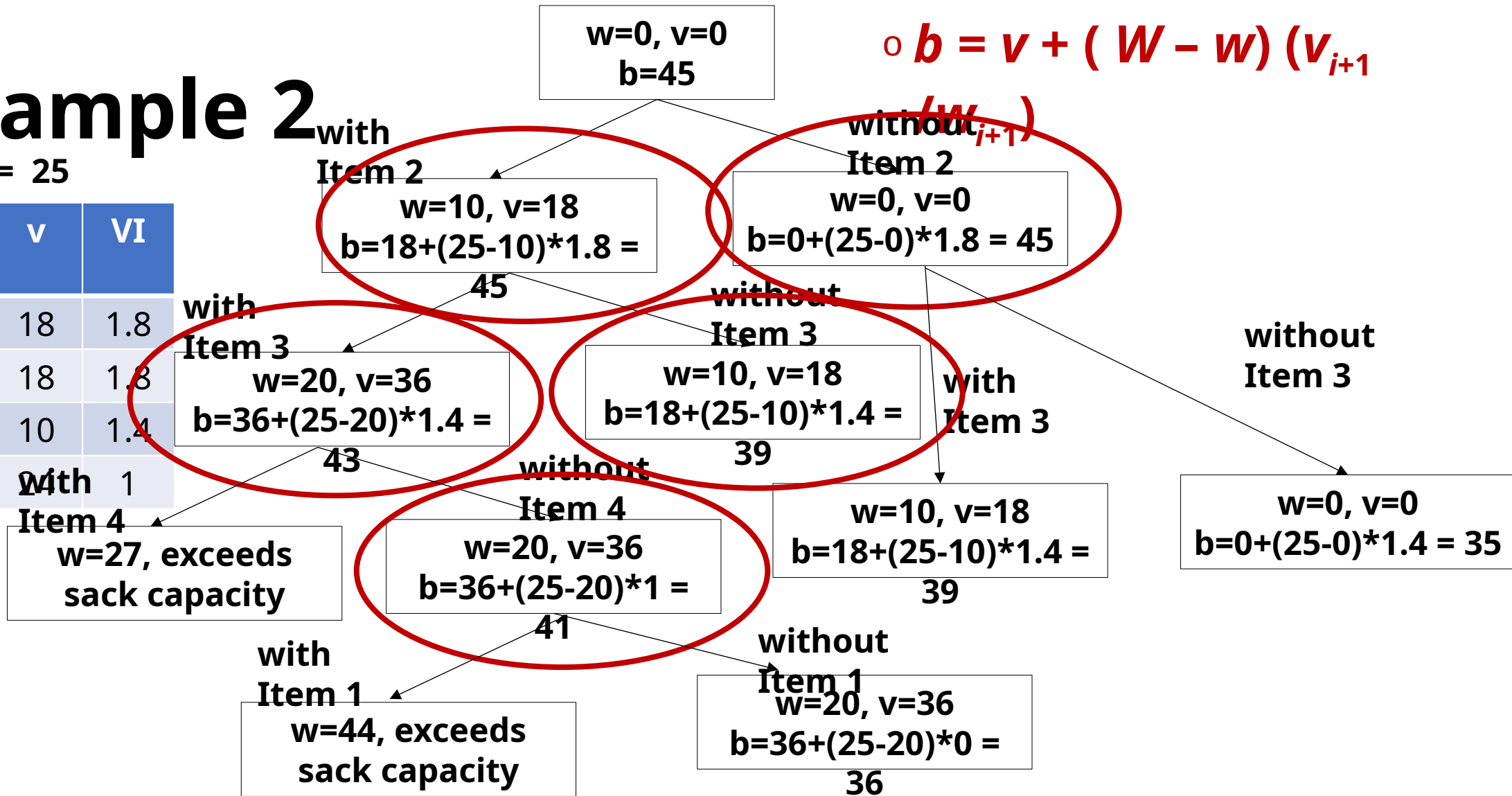
Item	Weight (KG)	Value (\$)	Value Index
1	24	24	1
2	10	18	1.8
3	10	18	1.8
4	7	10	1.4

Example 2

W= 25

Item	w	v	VI
2	10	18	1.8
3	10	18	1.8
4	7	10	1.4
1	24	0	0

Sorted
based on
Value
Index

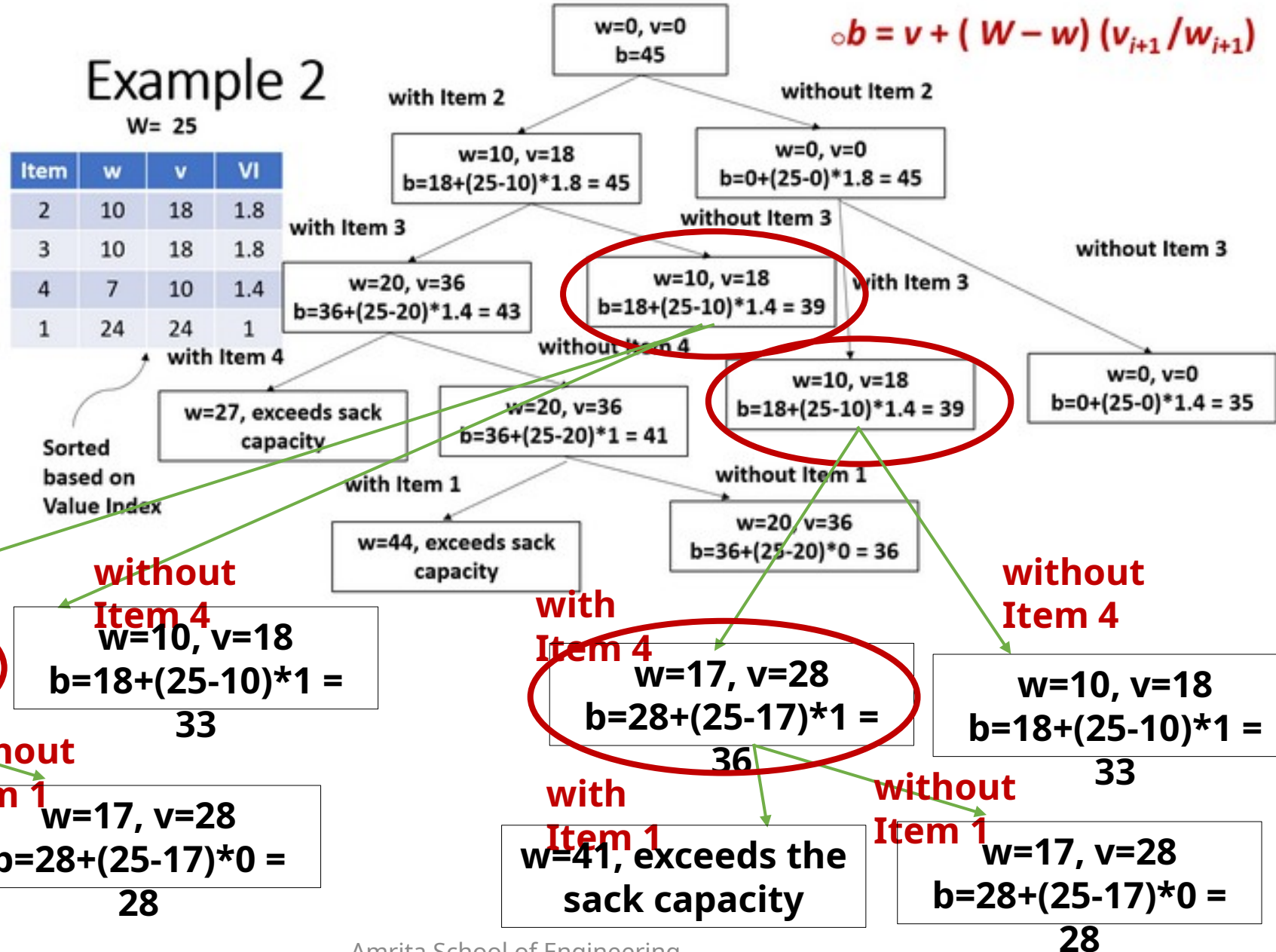


Example ~ --contd

Example 2

W= 25

Item	w	v	VI
2	10	18	1.8
3	10	18	1.8
4	7	10	1.4
1	24	24	1



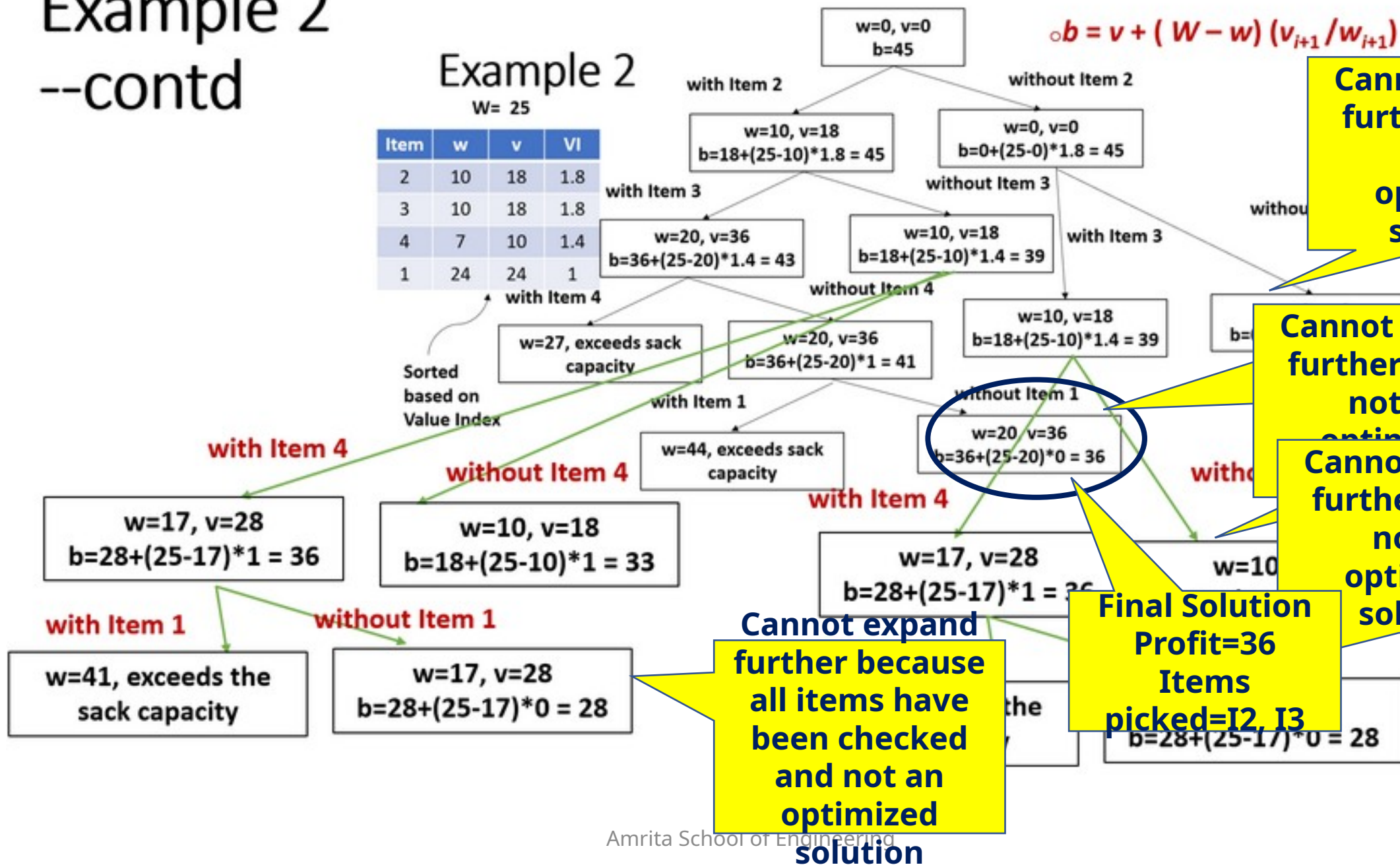
Example 2 --contd

Example 2

W= 25

Item	w	v	VI
2	10	18	1.8
3	10	18	1.8
4	7	10	1.4
1	24	24	1

Sorted
based on
Value Index



Example 3

Try this as an assignment

- Listed below are some semi-precious stones available for one to pick up. Solve the problem using the Branch and Bound approach, such that the knapsack has a profit limit over the chosen objects to be no more than 6, and you are to fill the knapsack with maximum possible total weight. You are not allowed to break any object. Show your work.

Object Id.	1	2	3	4
Weights	6	3	4	4
Profits	2	3	2	1

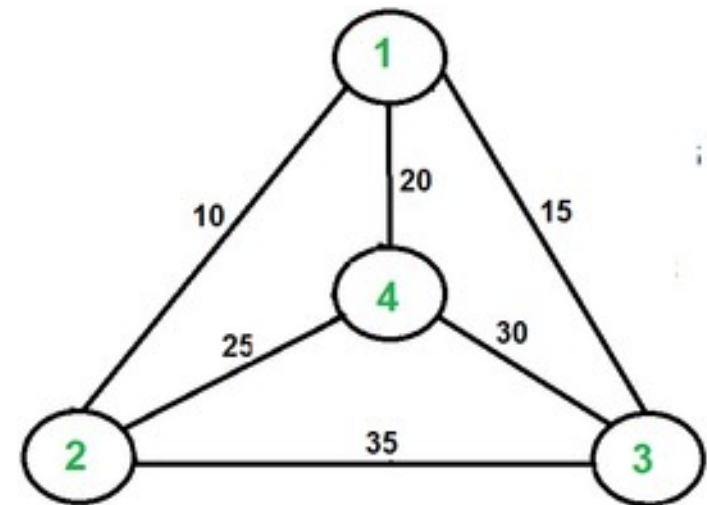
Travelling Salesman Problem (TSP)

- Given a set of cities and distance between every pair of cities, the problem is to find the shortest possible route that visits every city exactly once and returns to the starting point.
- For example, consider the graph shown in figure on right side. A TSP tour in the graph is 1-2-4-3-1. The cost of the tour is $10+25+30+15$ which is 80.
- The problem is a famous NP hard problem.
There is no polynomial time solution for this problem.

Complexity using Brute Force – $\theta(n!)$

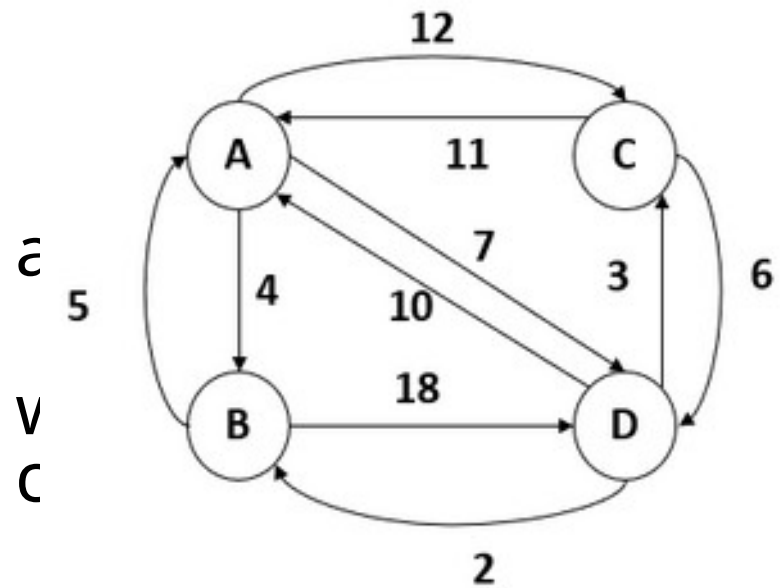
Complexity using DP – $O(n^2 2^n)$ – exponential

Complexity using Branch and Bound – $O(n!)$,
but in best case works better



Example 1

- Solve the given TSP using Branch and Bound



and

Solution:

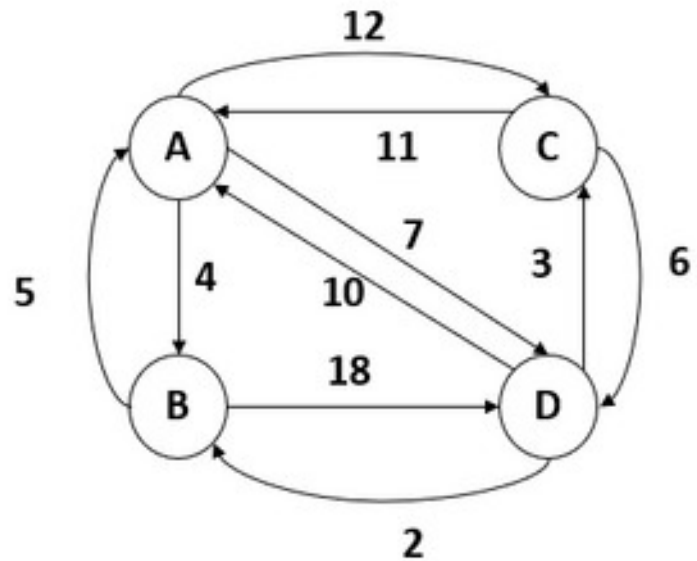
Step 1: Write the initial cost matrix
reduce it

Step 2: Select the best vertex where
can land upon to minimize the tour

Step 3: Expand the selected vertex
repeat step 1 till the end point is
reached

Example 1: TSP

- Step 1: Write the initial cost matrix and reduce it



Initial Cost Matrix

	A	B	C	D
A	∞	4	12	7
B	5	∞	∞	18
C	11	∞	∞	6
D	10	2	3	∞

- For reduction, do the following:
 - Reduce by row and reduce by column
 - Subtract each row and column by the minimum entry in that row/column
 - If a particular row or a column has a zero (0) then that row/column need not be reduced (i.e.) it is already in the reduced form

Example 1: TSP -- contd

	A	B	C	D	Min
A	∞	4	12	7	4
B	5	∞	∞	18	5
C	11	∞	∞	6	6
D	10	2	3	∞	2

Reduce
by row



	A	B	C	D
A	∞	0	8	3
B	0	∞	∞	13
C	5	∞	∞	0
D	8	0	1	∞
Min	--	--	1	--

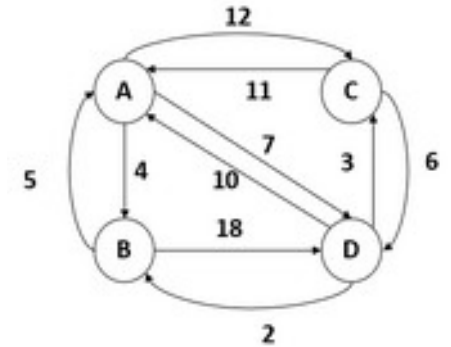
Reduce by
column

	A	B	C	D
A	∞	0	7	3
B	0	∞	∞	13
C	5	∞	∞	0
D	8	0	0	∞

Cost of Node 1 (Initial Node) = Previous node Cost +
Min of

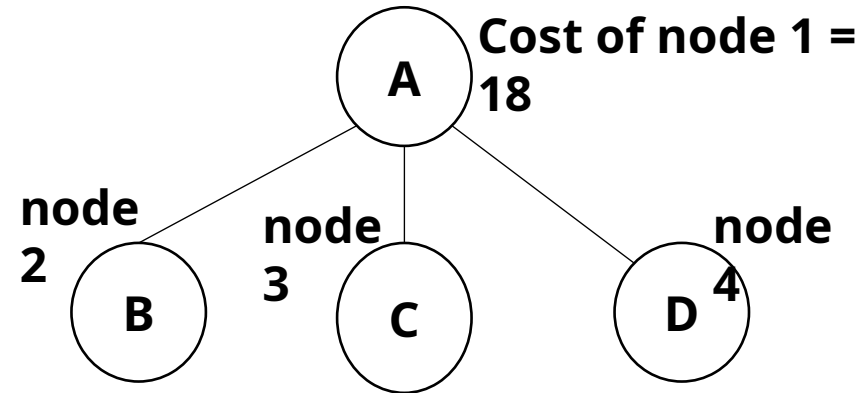
$$\text{Row} + \text{Min of Col} = 0 + 4+5+6+2+1 = 18$$

Example 1: TSP -- contd



- State space tree

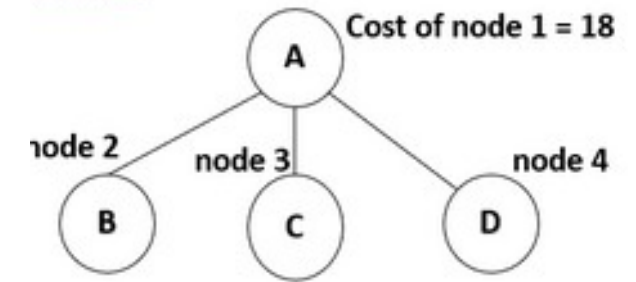
- Name the nodes with unique numbers as and when explored (i.e.) root A will be called as node 1
- We have the cost of the first node 1 computed in the previous step as 18
- Now proceed with the next level construction in the tree
- Explore the possibilities or options for the salesman to travel
- Find the cost of all the possibilities from the previously reduced matrix
- Choose the node with the minimum cost to explore further and repeat the steps



To start with the next step:

- Explore the next possible node, say A--B in the tree
- Consider the matrix obtained in the step 1 and note the value of matrix[A,B] which is 0 (Zero) for further computation
- Set entire Row A and Column B to ∞
- Set matrix[B,A] to ∞
- Now write the new cost matrix with these updates
- Go with row reduction and column reduction mentioned in the previous slide and compute the cost of the node
- Now repeat these steps for the next possible nodes and choose the node with minimum cost for the next

Example 1: TSP -- contd



- Now exploring A—B path
 - The cost matrix of node A becomes...

	A	B	C	D
A	∞	0	7	3
B	0	∞	∞	13
C	5	∞	∞	0
D	8	0	0	∞



	A	B	C	D	Min
A	∞	∞	∞	∞	--
B	∞	∞	∞	13	13
C	5	∞	∞	0	--
D	8	∞	0	∞	--

Reduce
by row



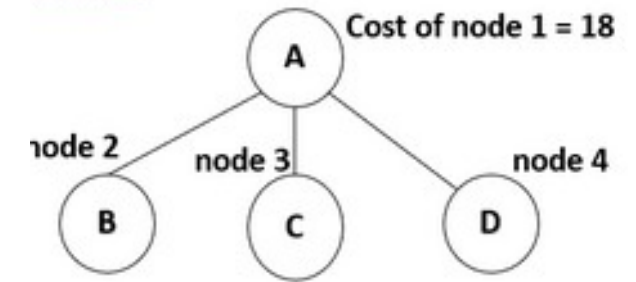
	A	B	C	D
A	∞	∞	∞	∞
B	∞	∞	∞	0
C	5	∞	∞	0
D	8	∞	0	∞
Min	5	--	--	--

Reduce by
column

	A	B	C	D
A	∞	∞	∞	∞
B	∞	∞	∞	0
C	0	∞	∞	0
D	3	∞	0	∞

Cost of Node 2= Previous node Cost + Min of Row +
Min of Col + cost of current link (i.e.) matrix[A,B]
= 18 + 13 + 5 + 0 = 36

Example 1: TSP -- contd



- Now exploring A—C path
 - The cost matrix of node A becomes...

	A	B	C	D
A	∞	0	7	3
B	0	∞	∞	13
C	5	∞	∞	0
D	8	0	0	∞



	A	B	C	D	Min
A	∞	∞	∞	∞	--
B	0	∞	∞	13	--
C	∞	∞	∞	0	--
D	8	0	∞	∞	--

Reduce
by row



	A	B	C	D
A	∞	∞	∞	∞
B	0	∞	∞	13
C	∞	∞	∞	0
D	8	0	∞	∞

Min

--

--

--

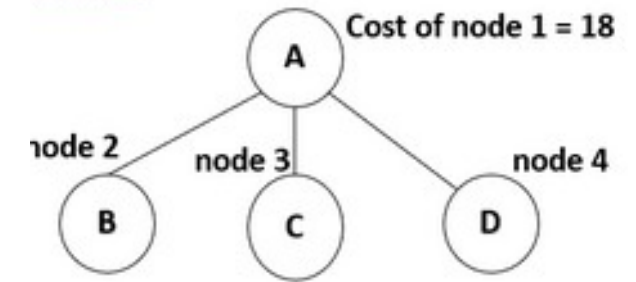
--

Reduce by
column

Cost of Node 3= Previous node Cost + Min of Row +
Min of Col + cost of current link (i.e.) matrix[A,C]
= 18 + 0 + 0 + 7 = 25

	A	B	C	D
A	∞	∞	∞	∞
B	0	∞	∞	13
C	∞	∞	∞	0
D	8	0	∞	∞

Example 1: TSP -- contd



- Now exploring A—D path
- The cost matrix of node A becomes...

Reduce
by row

	A	B	C	D
A	∞	0	7	3
B	0	∞	∞	13
C	5	∞	∞	0
D	8	0	0	∞



	A	B	C	D	Min
A	∞	∞	∞	∞	--
B	0	∞	∞	∞	--
C	5	∞	∞	∞	5
D	∞	0	0	∞	--

	A	B	C	D
A	∞	∞	∞	∞
B	0	∞	∞	∞
C	0	∞	∞	∞
D	∞	0	0	∞

-- -- -- --

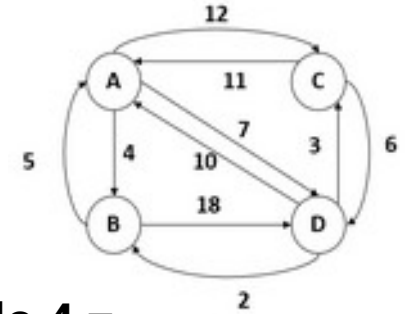
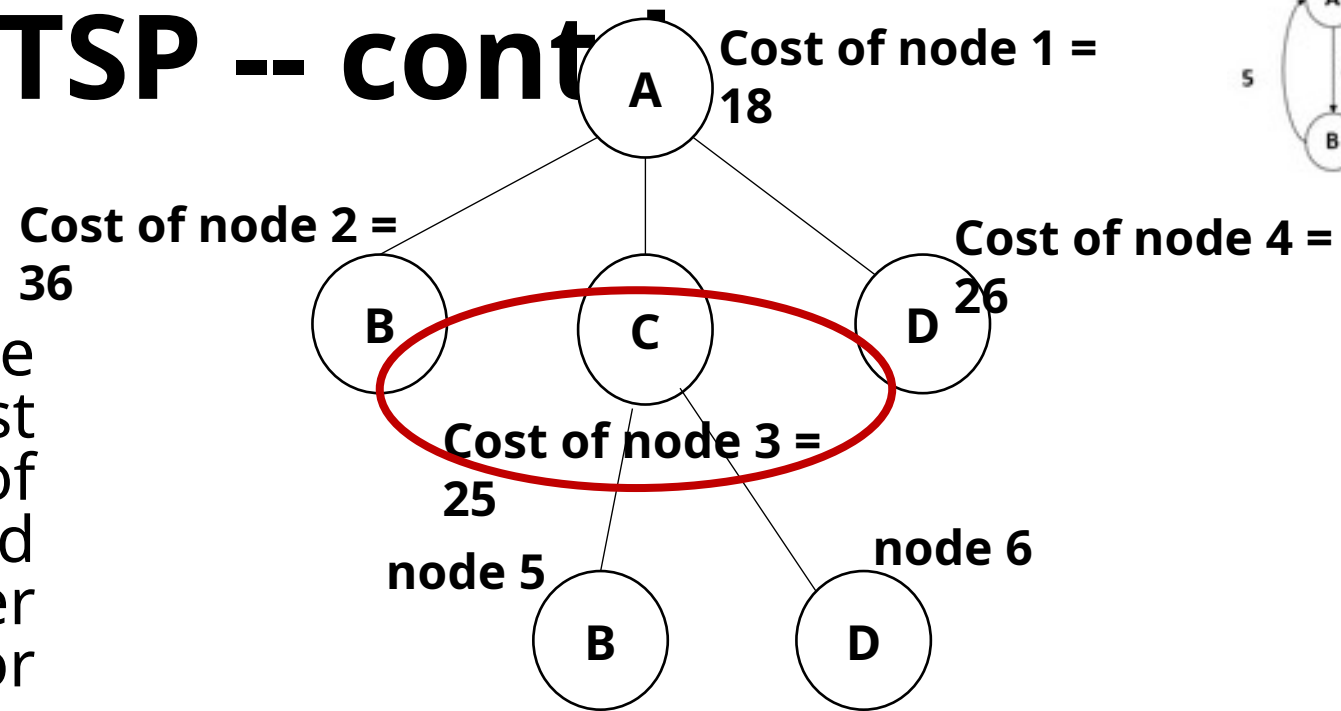
Reduce by
column

	A	B	C	D
A	∞	∞	∞	∞
B	0	∞	∞	∞
C	0	∞	∞	∞
D	∞	0	0	∞

Cost of Node 4 = Previous node Cost + Min of Row +
Min of Col + cost of current link (i.e.) matrix[A,D]
= 18 + 5 + 0 + 3 = 26

Example 1: TSP -- cont

- From the resultant tree choose the lowest cost node for the next level of exploration and proceed with the further remaining options for the salesman to explore
- In this case, explore node 3 and find the cost of each node in the next level
- Repeat the previous steps



Example 1: TSP -- contd

- Now exploring A—C—B path
 - The cost matrix of node C becomes...

Make $\text{mat}[B, A] = \infty$

	A	B	C	D
A	∞	∞	∞	∞
B	0	∞	∞	13
C	∞	∞	∞	0
D	8	0	∞	∞



	A	B	C	D	Min
A	∞	∞	∞	∞	--
B	∞	∞	∞	13	13
C	∞	∞	∞	∞	--
D	8	∞	∞	∞	8

Reduce
by row



	A	B	C	D
A	∞	∞	∞	∞
B	∞	∞	∞	0
C	∞	∞	∞	∞
D	0	∞	∞	∞

Min

--

--

--

--

--

--

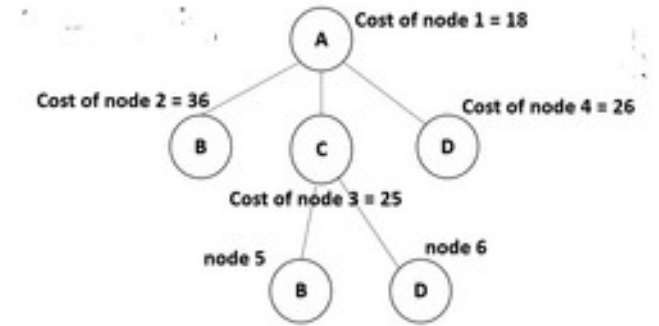
--

--

--

Reduce by
column

Cost of Node 5 = Previous node Cost + Min of Row +
Min of Col + cost of current link (i.e.) $\text{matrix}[C, B]$
 $= 25 + 8 + 13 + \infty = \infty$



	A	B	C	D
A	∞	∞	∞	∞
B	∞	∞	∞	0
C	∞	∞	∞	∞
D	0	∞	∞	∞

Example 1: TSP -- contd

- Now exploring A — C—D path
 - The cost matrix of node C becomes...

	A	B	C	D
A	∞	∞	∞	∞
B	0	∞	∞	13
C	∞	∞	∞	0
D	8	0	∞	∞



	A	B	C	D	Min
A	∞	∞	∞	∞	--
B	0	∞	∞	∞	--
C	∞	∞	∞	∞	--
D	∞	0	∞	∞	--

Reduce
by row

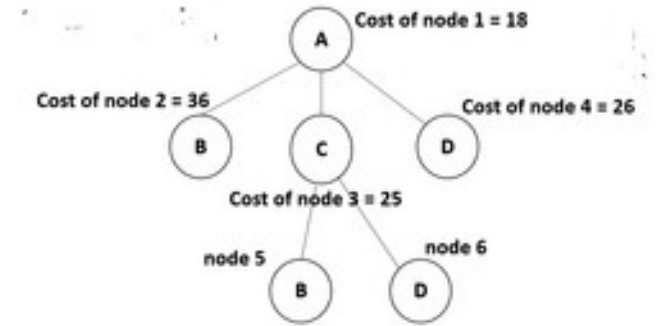


	A	B	C	D
A	∞	∞	∞	∞
B	0	∞	∞	∞
C	∞	∞	∞	∞
D	∞	0	∞	∞
Min	--	--	--	--

Reduce by
column

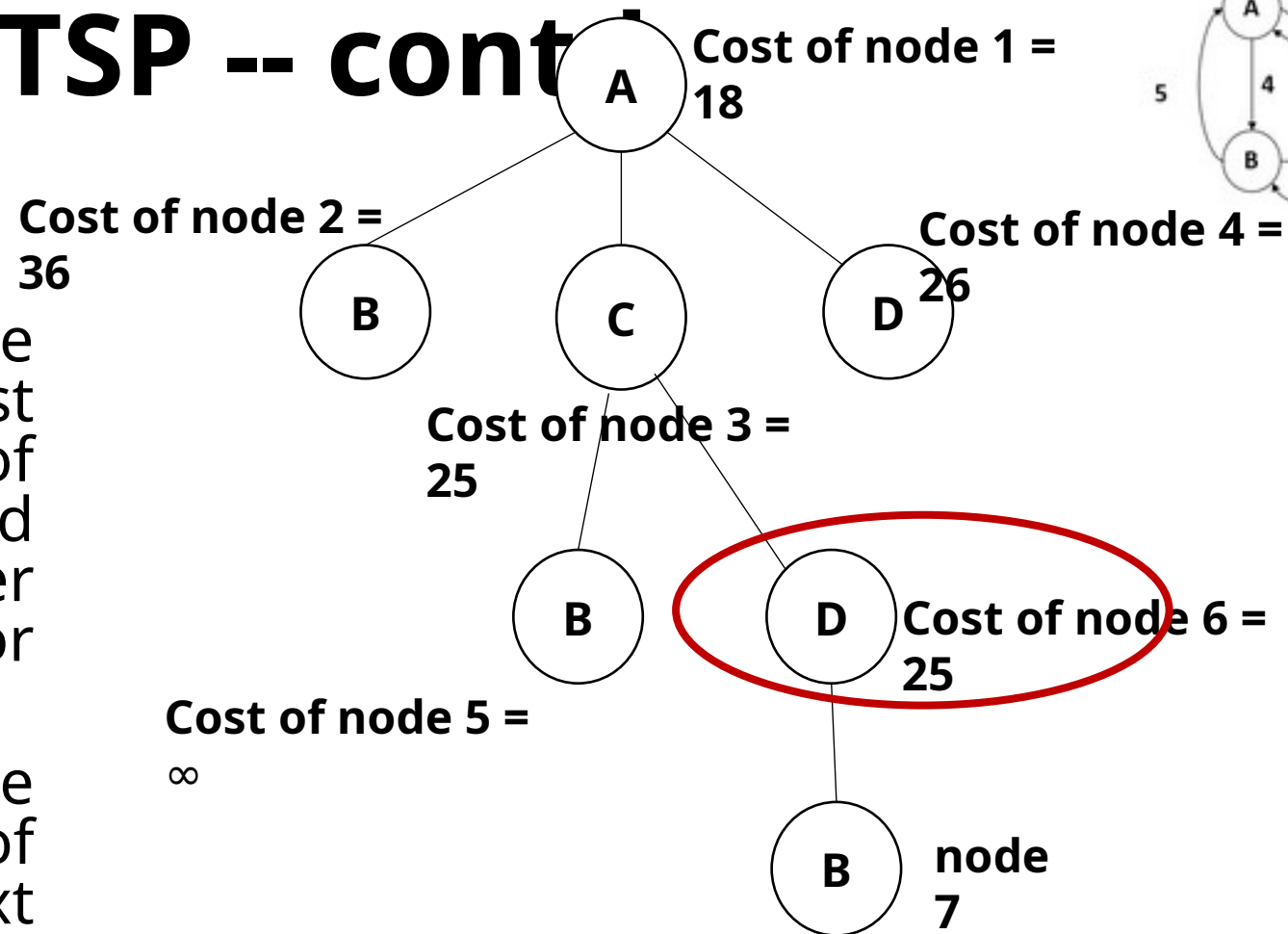
	A	B	C	D
A	∞	∞	∞	∞
B	0	∞	∞	∞
C	∞	∞	∞	∞
D	∞	0	∞	∞

Cost of Node 6 = Previous node Cost + Min of Row +
Min of Col + cost of current link (i.e.) matrix[C,D]
= 25 + 0 + 0 + 0 = 25



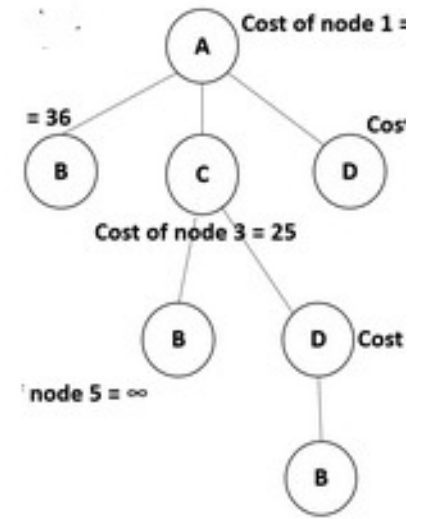
Example 1: TSP -- cont

- From the resultant tree choose the lowest cost node for the next level of exploration and proceed with the further remaining options for the salesman to explore
- In this case, explore node 6 and find the cost of each node in the next level
- Repeat the previous steps



Example 1: TSP -- contd

- Now exploring A — C—D —B path
 - The cost matrix of node D becomes...



Reduce by row

	A	B	C	D
A	∞	∞	∞	∞
B	0	∞	∞	∞
C	∞	∞	∞	∞
D	∞	0	∞	∞

	A	B	C	D	Min
A	∞	∞	∞	∞	--
B	∞	∞	∞	∞	--
C	∞	∞	∞	∞	--
D	∞	∞	∞	∞	--

Cost of Node 7 = Previous node Cost + Min of Row + Min of Col + cost of current link (i.e.) matrix[D,B]
= 25 + 0 + 0 + 0 = 25

Node is completely

completely
reduced... ,
reached terminal

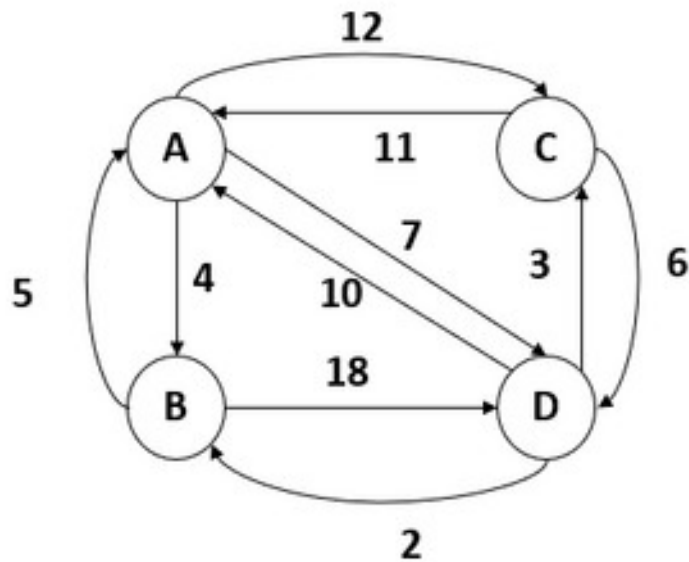
	A	B	C	D
A	∞	∞	∞	∞
B	∞	∞	∞	∞
C	∞	∞	∞	∞
D	∞	∞	∞	∞

Min -- -- --  --

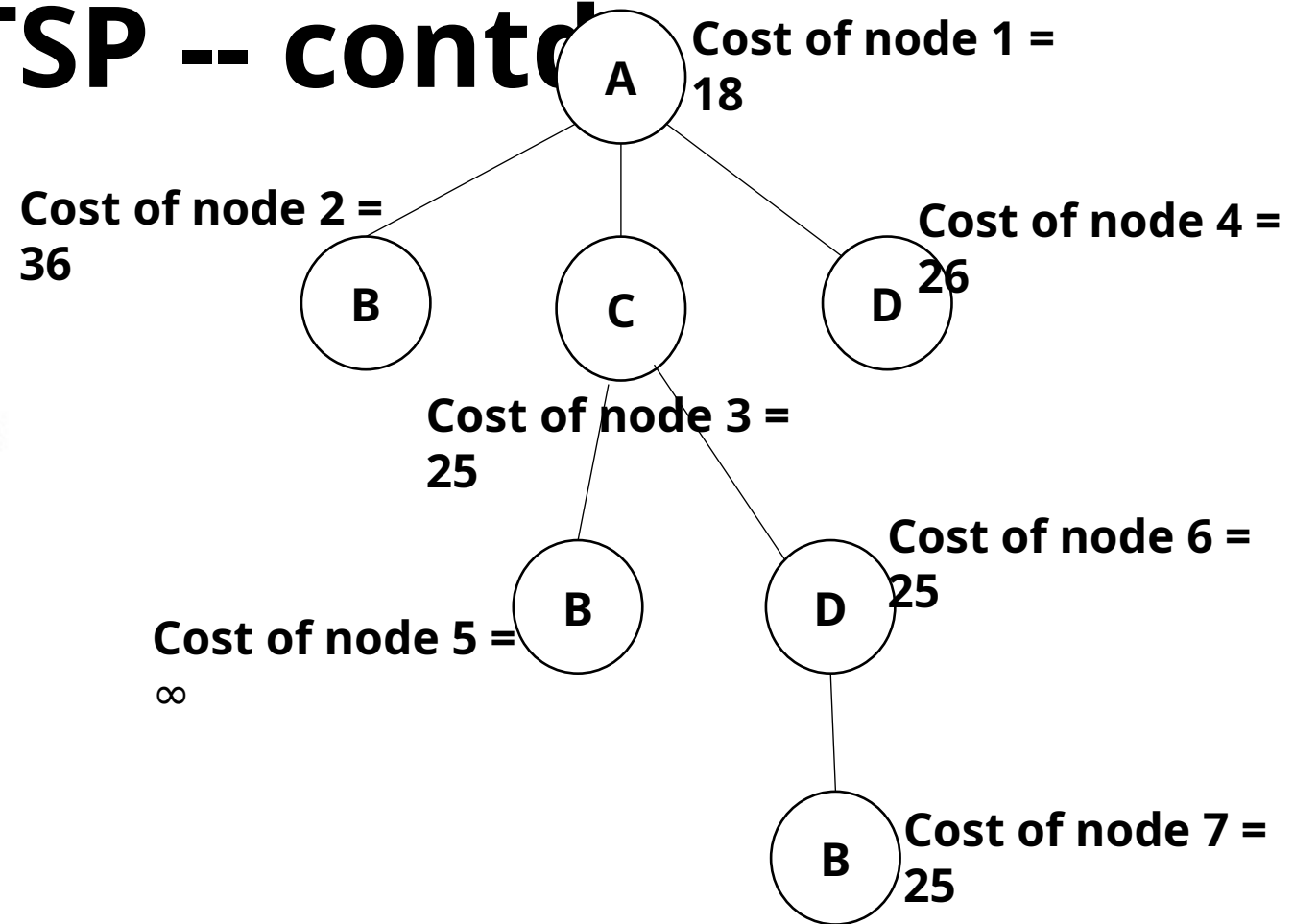
Reduce by column

	A	B	C	D
A	∞	∞	∞	∞
B	∞	∞	∞	∞
C	∞	∞	∞	∞
D	∞	∞	∞	∞

Example 1: TSP -- contd

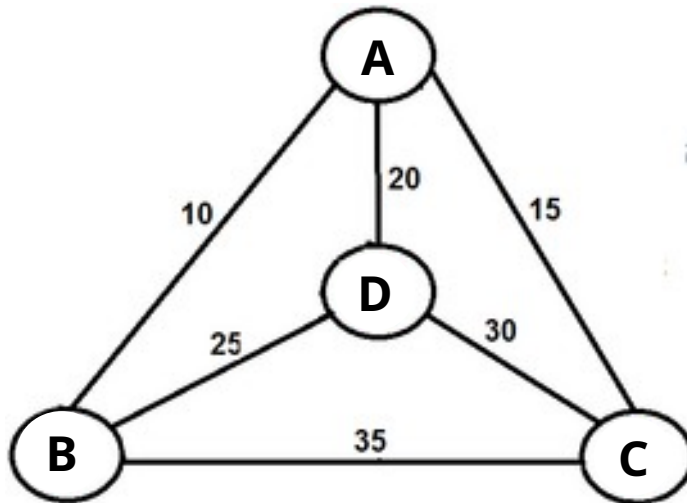


Final Solution (Path) is
A - C - D - B - A
with minimum cost as
25



Example 2: Solve the given TSP

- Step 1: Write the initial cost matrix and reduce it



Initial Cost Matrix

	A	B	C	D
A	∞	10	15	20
B	10	∞	35	25
C	15	35	∞	30
D	20	25	30	∞

- For reduction, do the following:
 - Reduce by row and reduce by column
 - Subtract each row and column by the minimum entry in that row/column
 - If a particular row or a column has a zero (0) then that row/column need not be reduced

Example 2: TSP -- contd

	A	B	C	D	Min
A	∞	10	15	20	10
B	10	∞	35	25	10
C	15	35	∞	30	15
D	20	25	30	∞	20

Reduce
by row



	A	B	C	D
A	∞	0	5	10
B	0	∞	25	15
C	0	20	∞	15
D	0	5	10	∞
Min	--	--	5	10

Reduce by
column

	A	B	C	D
A	∞	0	0	0
B	0	∞	20	5
C	0	20	∞	5
D	0	5	5	∞

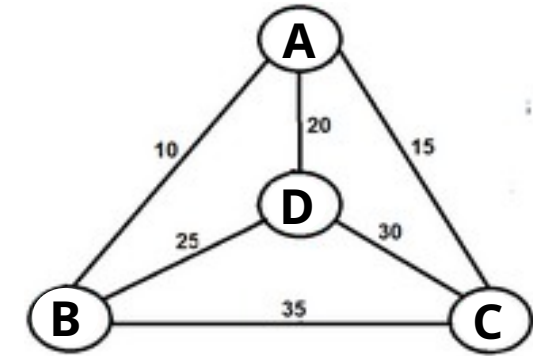
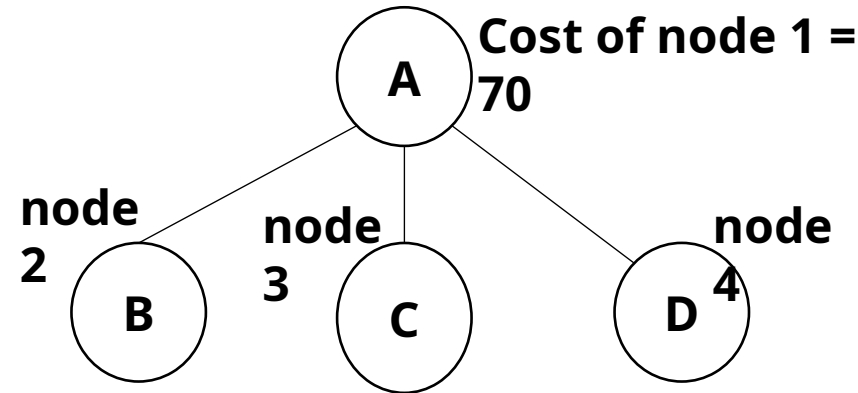
Cost of Node 1 (Initial Node) = Previous node Cost +
Min of

$$\text{Row} + \text{Min of Col} = 0 + 55 + 15 = 70$$

Example 2: TSP -- contd

- State space tree

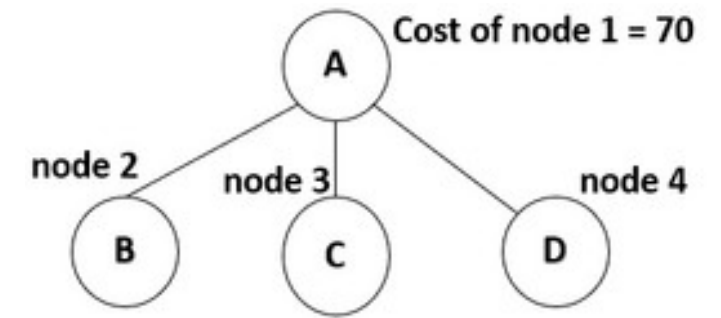
- Name the nodes with unique numbers as and when explored (i.e.) root A will be called as node 1
- We have the cost of the first node 1 computed in the previous step as 70
- Now proceed with the next level construction in the tree
- Explore the possibilities or options for the salesman to travel
- Find the cost of all the possibilities from the previously reduced matrix
- Choose the node with the minimum cost to explore further and repeat the steps



To start with the next step:

- Explore the next possible node, say A--B in the tree
- Consider the matrix obtained in the step 1 and note the value of matrix[A,B] which is 0 (Zero) for further computation
- Set entire Row A and Column B to ∞
- Set matrix[B,A] to ∞
- Now write the new cost matrix with these updates
- Go with row reduction and column reduction mentioned in the previous slide and compute the cost of the node

Example 2 : TSP -- contd



- Now exploring A—B path
 - The cost matrix of node A becomes...

	A	B	C	D
A	∞	0	0	0
B	0	∞	20	5
C	0	20	∞	5
D	0	5	5	∞



	A	B	C	D	Min
A	∞	∞	∞	∞	--
B	∞	∞	20	5	5
C	0	∞	∞	5	--
D	0	∞	5	∞	--

Reduce
by row



	A	B	C	D
A	∞	∞	∞	∞
B	∞	∞	15	0
C	0	∞	∞	5
D	0	∞	5	∞

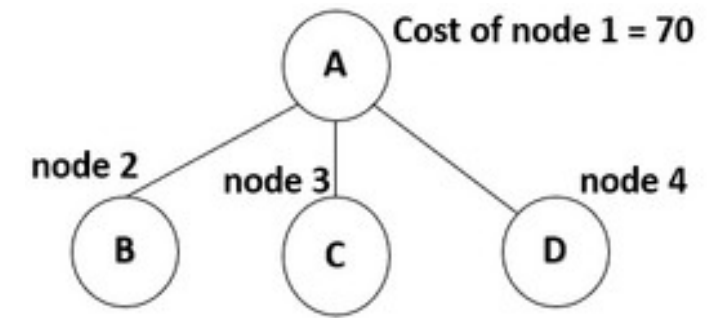
Min	--	--	5	--
-----	----	----	---	----

Reduce by
column

	A	B	C	D
A	∞	∞	∞	∞
B	∞	∞	10	0
C	0	∞	∞	5
D	0	∞	0	∞

Cost of Node 2= Previous node Cost + Min of Row +
Min of Col + cost of current link (i.e.) matrix[A,B]
= 70 + 5 + 5 + 0 = 80

Example 2 : TSP -- contd



- Now exploring A—C path
 - The cost matrix of node A becomes...

	A	B	C	D
A	∞	0	0	0
B	0	∞	20	5
C	0	20	∞	5
D	0	5	5	∞



	A	B	C	D	Min
A	∞	∞	∞	∞	--
B	0	∞	∞	5	--
C	∞	20	∞	5	5
D	0	5	∞	∞	--

Reduce
by row



	A	B	C	D
A	∞	∞	∞	∞
B	0	∞	∞	5
C	∞	15	∞	0
D	0	5	∞	∞

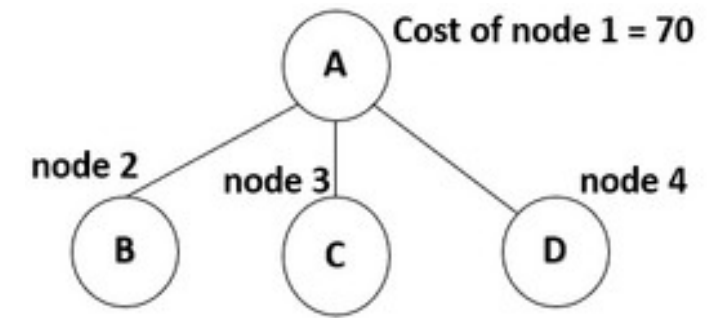
Min

Reduce by
column

	A	B	C	D
A	∞	∞	∞	∞
B	0	∞	∞	5
C	∞	10	∞	0
D	0	0	∞	∞

Cost of Node 3= Previous node Cost + Min of Row +
Min of Col + cost of current link (i.e.) matrix[A,C]
= 70 + 5 + 5 + 0 = 80

Example 2 : TSP -- contd



- Now exploring A—D path
 - The cost matrix of node A becomes...

Reduce
by row

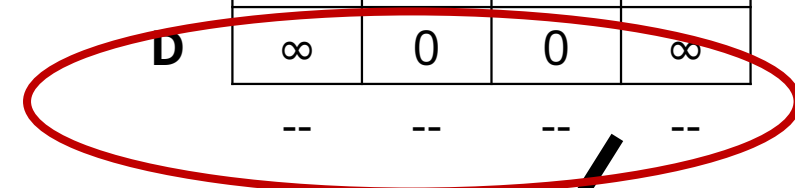
	A	B	C	D
A	∞	0	0	0
B	0	∞	20	5
C	0	20	∞	5
D	0	5	5	∞



	A	B	C	D	Min
A	∞	∞	∞	∞	--
B	0	∞	20	∞	--
C	0	20	∞	∞	--
D	∞	5	5	∞	5



	A	B	C	D
A	∞	∞	∞	∞
B	0	∞	20	∞
C	0	20	∞	∞
D	∞	0	0	∞



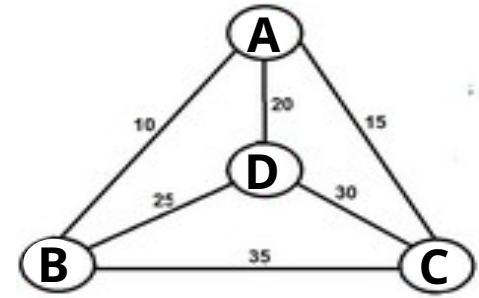
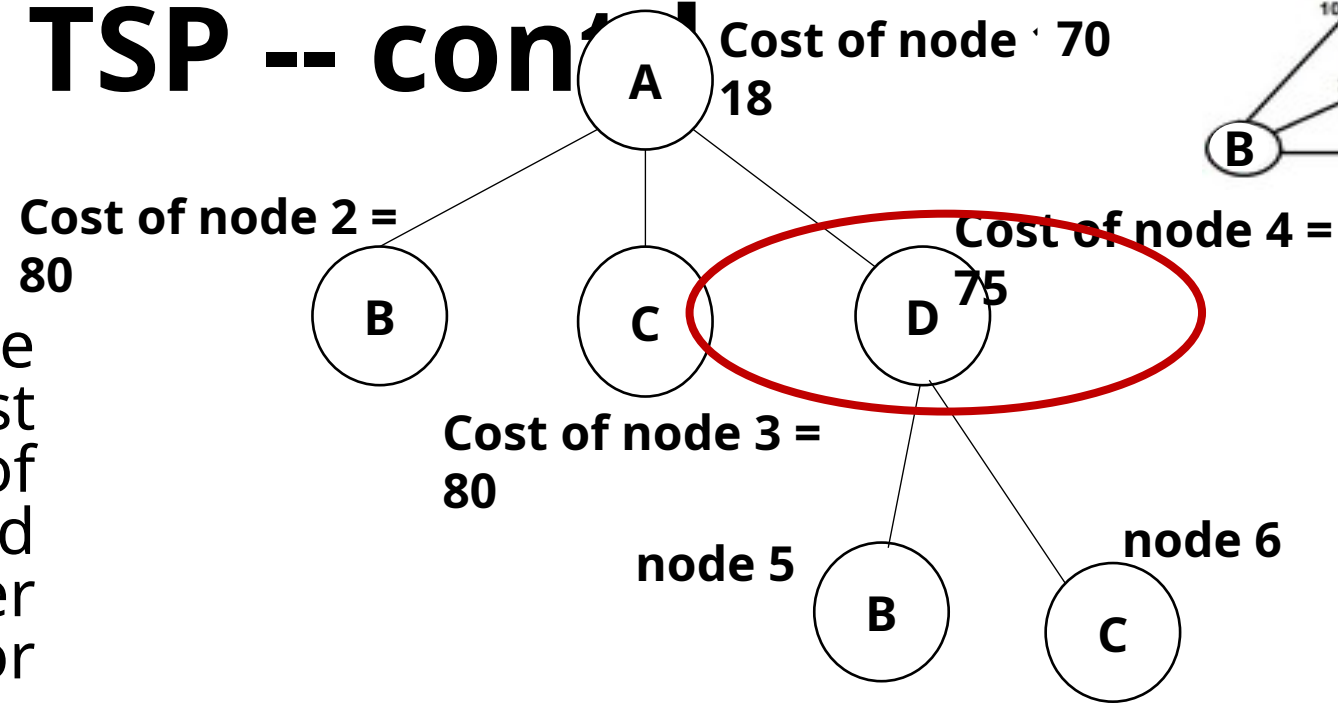
Reduce by
column

	A	B	C	D
A	∞	∞	∞	∞
B	0	∞	20	∞
C	0	20	∞	∞
D	∞	0	0	∞

Cost of Node 4= Previous node Cost + Min of Row +
Min of Col + cost of current link (i.e.) matrix[A,D]
= 70 + 5 + 0 + 0 = 75

Example 2 : TSP -- contd

- From the resultant tree choose the lowest cost node for the next level of exploration and proceed with the further remaining options for the salesman to explore
- In this case, explore node 4 and find the cost of each node in the next level
- Repeat the previous steps



Example 2 : TSP -- contd

- Now exploring A—D—B path
 - The cost matrix of node D becomes...

	A	B	C	D
A	∞	∞	∞	∞
B	0	∞	20	∞
C	0	20	∞	∞
D	∞	0	0	∞



	A	B	C	D	Min
A	∞	∞	∞	∞	--
B	∞	∞	20	∞	20
C	0	∞	∞	∞	--
D	∞	∞	∞	∞	--

Reduce
by row



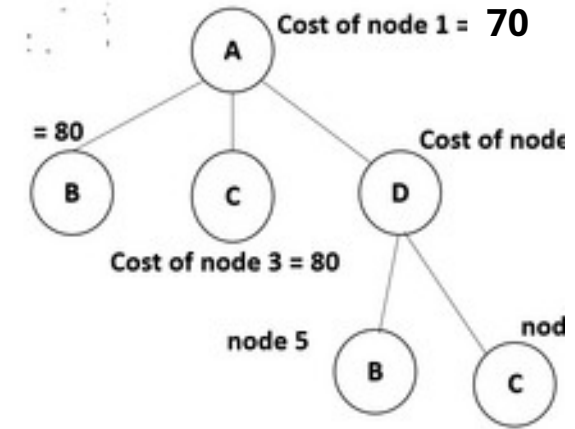
	A	B	C	D
A	∞	∞	∞	∞
B	∞	∞	0	∞
C	0	∞	∞	∞
D	∞	∞	∞	∞

Min

Reduce by
column

	A	B	C	D
A	∞	∞	∞	∞
B	∞	∞	0	∞
C	0	∞	∞	∞
D	∞	∞	∞	∞

Cost of Node 5 = Previous node Cost + Min of Row +
Min of Col + cost of current link (i.e.) matrix[D,B]
= 75 + 20 + 0 + 0 = 95



Example 2 : TSP -- contd

- Now exploring A — D—C path
 - The cost matrix of node D becomes...

	A	B	C	D
A	∞	∞	∞	∞
B	0	∞	20	∞
C	0	20	∞	∞
D	∞	0	0	∞



	A	B	C	D	Min
A	∞	∞	∞	∞	--
B	0	∞	∞	∞	--
C	∞	20	∞	∞	20
D	∞	∞	∞	∞	--

Reduce
by row



	A	B	C	D
A	∞	∞	∞	∞
B	0	∞	∞	∞
C	∞	0	∞	∞
D	∞	∞	∞	∞

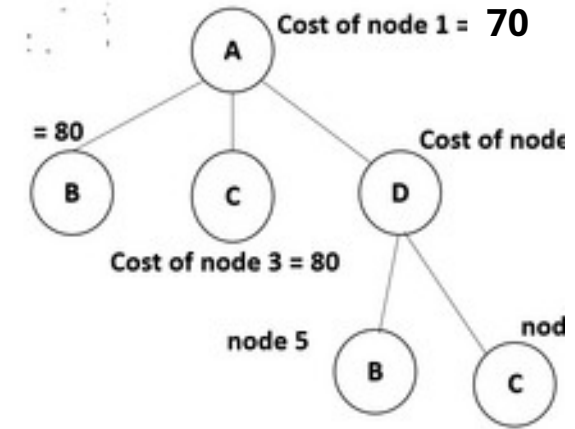
Min

-- -- -- --

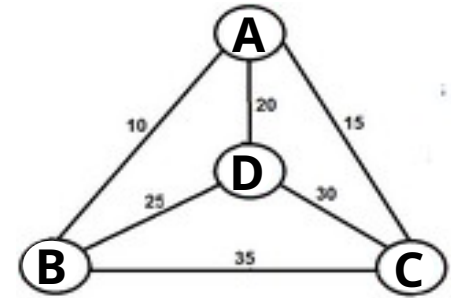
Reduce by
column

	A	B	C	D
A	∞	∞	∞	∞
B	0	∞	∞	∞
C	∞	0	∞	∞
D	∞	∞	∞	∞

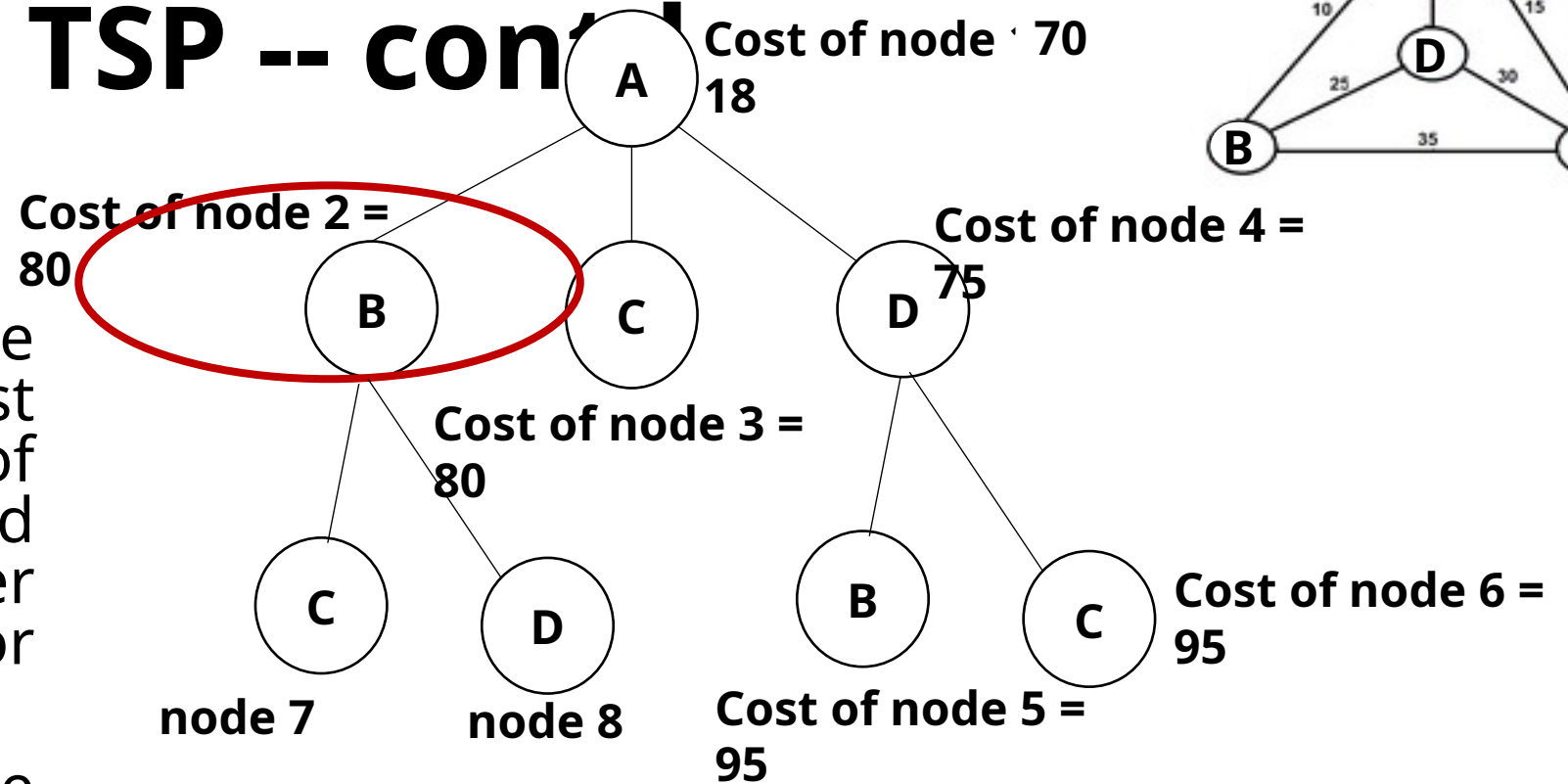
Cost of Node 6= Previous node Cost + Min of Row +
Min of Col + cost of current link (i.e.) matrix[D,C]
= 75 + 20 + 0 + 0 = 95



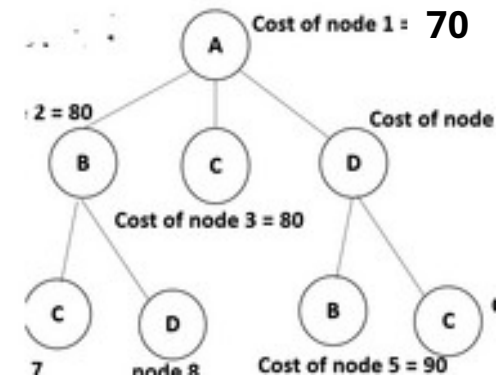
Example 2 : TSP -- continued



- From the resultant tree choose the lowest cost node for the next level of exploration and proceed with the further remaining options for the salesman to explore
- In this case, explore node 2 and find the cost of each node in the next level
- Repeat the previous steps



Example 2 : TSP -- contd



- Now exploring A—B—C path
 - The cost matrix of node B becomes...

	A	B	C	D
A	∞	∞	∞	∞
B	∞	∞	10	0
C	0	∞	∞	5
D	0	∞	0	∞



	A	B	C	D	Min
A	∞	∞	∞	∞	--
B	∞	∞	∞	∞	--
C	∞	∞	∞	5	5
D	0	∞	∞	∞	--

Reduce
by row



	A	B	C	D
A	∞	∞	∞	∞
B	∞	∞	∞	∞
C	∞	∞	∞	0
D	0	∞	∞	∞

Min

Reduce by
column

	A	B	C	D
A	∞	∞	∞	∞
B	∞	∞	∞	∞
C	∞	∞	∞	0
D	0	∞	∞	∞

Cost of Node 7 = Previous node Cost + Min of Row +
Min of Col + cost of current link (i.e.) matrix[B,C]
= 80 + 5 + 0 + 10 = 95

Example 2 : TSP -- contd

- Now exploring A — B—D path
 - The cost matrix of node B becomes...

	A	B	C	D
A	∞	∞	∞	∞
B	∞	∞	10	0
C	0	∞	∞	5
D	0	∞	0	∞



	A	B	C	D	Min
A	∞	∞	∞	∞	--
B	∞	∞	∞	∞	--
C	0	∞	∞	∞	--
D	∞	∞	0	∞	--

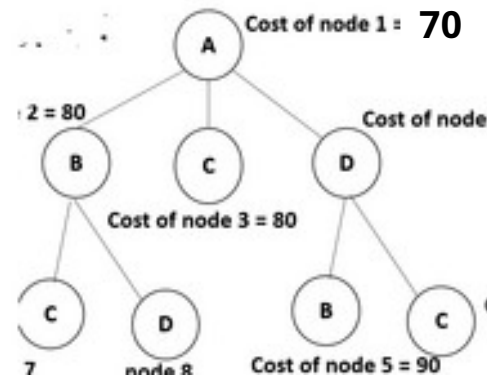
Reduce by row

	A	B	C	D
A	∞	∞	∞	∞
B	∞	∞	∞	∞
C	0	∞	∞	∞
D	∞	∞	0	∞
Min	--	--	--	--

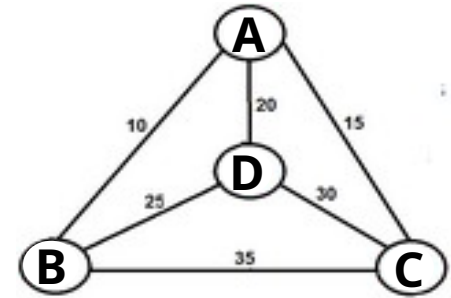
Reduce by column

	A	B	C	D
A	∞	∞	∞	∞
B	∞	∞	∞	∞
C	0	∞	∞	∞
D	∞	∞	0	∞

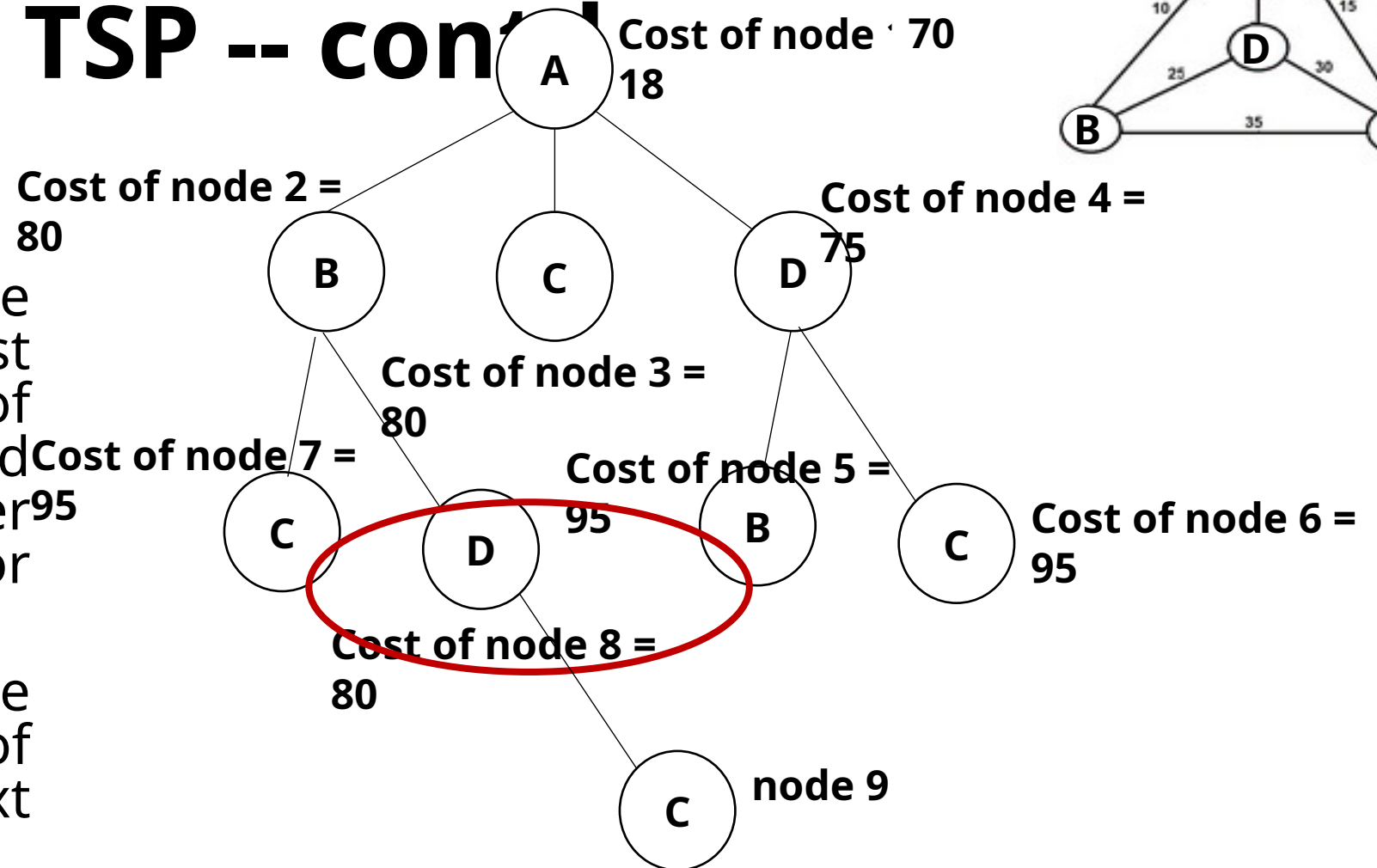
Cost of Node 8= Previous node Cost + Min of Row + Min of Col + cost of current link (i.e.) matrix[B,D]
 = 80 + 0 + 0 + 0 = 80



Example 2 : TSP -- continued



- From the resultant tree choose the lowest cost node for the next level of exploration and proceed with the further remaining options for the salesman to explore
- In this case, explore node 8 and find the cost of each node in the next level
- Repeat the previous steps



Example 2 : TSP -- contd

- Now exploring A — B—D —C path
 - The cost matrix of node D becomes...

Reduce
by row

	A	B	C	D
A	∞	∞	∞	∞
B	∞	∞	∞	∞
C	0	∞	∞	∞
D	∞	∞	0	∞



	A	B	C	D	Min
A	∞	∞	∞	∞	--
B	∞	∞	∞	∞	--
C	∞	∞	∞	∞	--
D	∞	∞	∞	∞	--



	A	B	C	D
A	∞	∞	∞	∞
B	∞	∞	∞	∞
C	∞	∞	∞	∞
D	∞	∞	∞	∞

Min

Min	--	--	--	--
-----	----	----	----	----

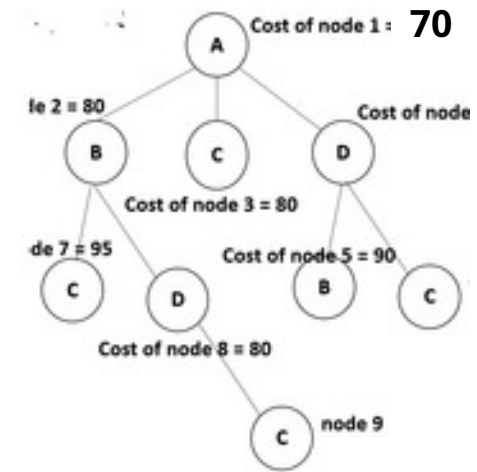
Reduce by
column

Cost of Node 9= Previous node Cost + Min of Row +
Min of Col + cost of current link (i.e.) matrix[D,C]
= 80 + 0 + 0 + 0 = 80

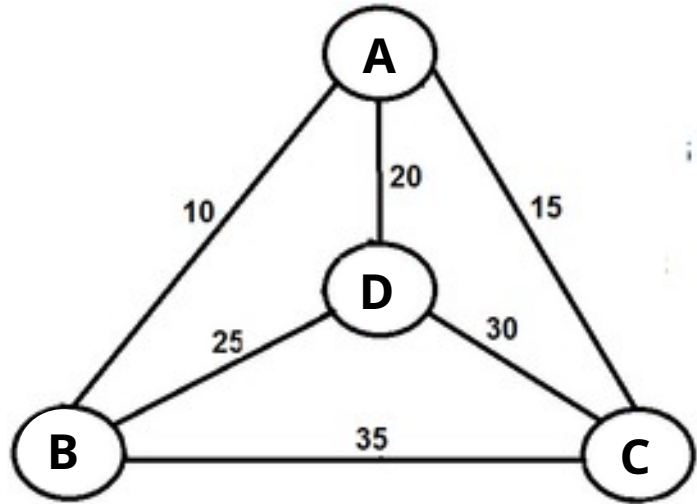
Node is
completely
reduced... ,
reached terminal

Engineering

	A	B	C	D
A	∞	∞	∞	∞
B	∞	∞	∞	∞
C	∞	∞	∞	∞
D	∞	∞	∞	∞



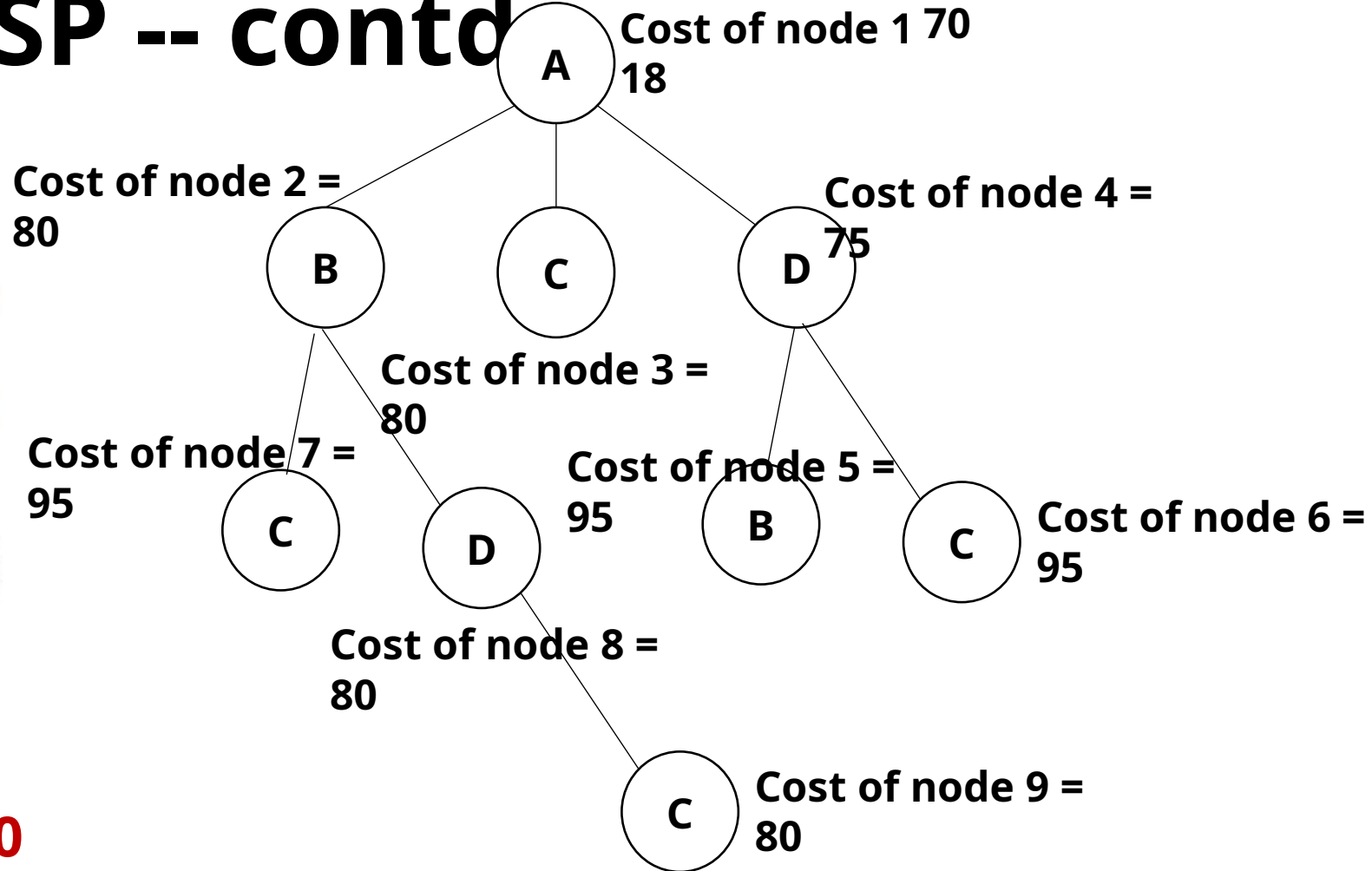
Example 2 : TSP -- contd



Final Solution (Path) is

A - B - D - C - A

with minimum cost as 80



Complexity of Branch and Bound

- These problems are typically exponential in terms of **time complexity** and may require exploring all possible permutations in worst case.
- But, Branch and Bound solve the problems relatively quickly (best case and average case).

Exercise Problem

- Solve the TSP given using Branch and Bound

